



UNIVERSIDAD AUTÓNOMA DE CAMPECHE

Facultad de Ingeniería

Práctica 01 - Sección Crítica.

Alumno:

Aarón David Chi Uc.

Profesor:

Ing. Sergio A Noh Puch

Ingeniería en Tecnología de Software

Campeche, Camp. — 2026

Ejecución de gcc P1_thread.c -lpthread -o programa

```
aaron@jarvis:/mnt/c/Users/aaron$ cd CyP_2026
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ ls
P1_thread.c  readme.txt
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ code .
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ gcc P1_thread.c -lpthread -o programa
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ ./programa 5
Starting thread...
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ cat readme
cat: readme: No such file or directory
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ cat readme.txt
New line
New line
New line
New line
New line
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ gcc P1_thread.c -lpthread -o programa
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ ./programa 100
Starting thread...
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ cat readme.txt
New line
New line
New line
New line
New line
New line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
Aaron Chi UcNew line
```

Concurrencia y paralelismo.

UACAM — Facultad de Ingeniería — 2026

Ejecución de gcc -S race_test.c

```
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ > readme.txt
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ cat readme.txt
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ code race_test.c
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ ls
P1_thread.c  programa  race_test.c  readme.txt
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ gcc -S race_test.c
race_test.c: In function 'main':
race_test.c:25:31: error: 'thread_routine_2' undeclared (first use in this function); did you mean 'thread_routine'?
   25 |         pthread_create(&t2, NULL, thread_routine_2, NULL);
      |                                   ^~~~~~
                                   thread_routine
race_test.c:25:31: note: each undeclared identifier is reported only once for each function it appears in
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ gcc -S race_test.c
aaron@jarvis:/mnt/c/Users/aaron/CyP_2026$ ls
P1_thread.c  programa  race_test.c  race_test.s  readme.txt
```

Análisis del archivo race_test.s

```
thread_routine:
.LFB0:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    movq    %rdi, -24(%rbp)
    movq    $0, -8(%rbp)
    jmp     .L2

.L3:
    movl    global_counter(%rip), %eax
    addl    $1, %eax
    movl    %eax, global_counter(%rip)
    addq    $1, -8(%rbp)

.L2:
    cmpq    $999, -8(%rbp)
    jbe     .L3
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc

.LFE0:
    .size   thread_routine, .-thread_routine
    .globl  thread_routine_2
    .type   thread_routine_2, @function
```

```
thread_routine_2:
.LFB1:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    movq    %rdi, -24(%rbp)
    movq    $0, -8(%rbp)
    jmp     .L6

.L7:
    movl    global_counter(%rip), %eax
    subl    $1, %eax
    movl    %eax, global_counter(%rip)
    addq    $1, -8(%rbp)

.L6:
    cmpq    $999, -8(%rbp)
    jbe     .L7
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc

.LFE1:
    .size   thread_routine_2, .-thread_routine_2
    .section .rodata

.LC0:
    .string "Global counter: %d\n"
    .text
    .globl  main
    .type   main, @function

main:
.LFB2:
    .cfi_startproc
    endbr64
    pushq   %rbp
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

^T Execute
^J Justify

SMART BOOK

Ayón David Chi Uc ITS

Concurrencia y Paralelismo

28/03/26

- Concepto de sección crítica.

Una sección crítica es una parte de un programa donde se accede a recursos compartidos, lo que requiere de mecanismos para evitar el acceso concurrente y garantizar la consistencia de los datos en entornos multihilo.

Es un concepto clave en la sincronización, que utiliza bloqueos o semáforos para permitir que solo un proceso o hilo ejecute la sección crítica a la vez.